1 Method

We introduce **GradMill**, a differentiable-simulation approach to CNC toolpath generation. Rather than learning a control policy, GradMill optimizes a discrete tool trajectory directly: a sequence of per-step cutter displacements is treated as the optimization variable, and gradients of a terminal geometry loss are backpropagated through a differentiable constructive-solid-geometry (CSG) carving simulator into the trajectory. The system is implemented in Taichi with `ti.ad.Tape` autodiff.

A central finding of this work is that the simulator's gradients optimize a *differentiable surrogate* whose optimum is misaligned with the *deployable* (boolean) machining outcome: the soft union over-erodes, so minimizing the soft loss systematically gouges the part under the exact boolean carve. Unlike prior work that sidesteps this by encoding the answer in a shape-specific initialization, our method is **shape-blind**: neither the optimizer, the initialization, nor the loss inspects or branches on the target shape name. The trajectory is initialized randomly and refined purely by gradient descent. The soft/hard misalignment is instead closed by three shape-blind mechanisms that act on the *surrogate itself*: (i) smoothness annealing (`k-anneal`) that aligns the training gradient with the sharp deployable metric, (ii) a de-biasing constant (`loss-shift`) that compensates the over-erosion so the soft loss targets the hard carve, and (iii) a tool-length / reachability correction that lets the optimizer discover the full part envelope. We describe each component below.

1.1 Problem Formulation

The machine work volume contains a rectilinear stock block (a normalized $[0,1]^3$ cube, default 1 in^3 at 0.5 mm voxels) and a target part S_{tgt} defined as the negative interior of an analytic signed distance field (SDF) $\phi_{\text{tgt}}(\mathbf{p})$. The target SDF may be a primitive (sphere, cylinder, box, square pyramid) or a *composite CSG* field: a through-hole sphere $S = S_{\text{sph}} \cap \neg S_{\text{cyl}}$, or a concave bowl $S = S_{\text{sph}} \cap \neg(S_{\text{sub}} \cap \{z \le z_c\})$. A cylindrical end mill of radius r and flute height h, surmounted coaxially by a wider holder (the collet/spindle), removes material by sweeping through the stock. A trajectory of T tool positions $\{\mathbf{x}_0, \ldots, \mathbf{x}_{T-1}\}$ produces a carved stock $S_{\text{carved}} \subseteq S_{\text{stock}}$. The objective is a trajectory whose carved geometry matches the target, scored by the Dice coefficient between the remaining material S_{carved} and the part S_{tgt}.

Shape-blindness constraint. The target shape is exposed to the simulator *only* as the pointwise SDF $\phi_{\text{tgt}}(\mathbf{p})$ used inside the loss. No component of the method — not the optimizer, the initialization, the loss, or any hyperparameter schedule — reads the shape label or branches on it. All hyperparameters reported in Sec. 2 are fixed across every shape, including the composite CSG targets. This is the operating constraint under which the method is evaluated: it must generalize to arbitrary, unseen shape combinations from geometry alone.

1.2 Differentiable CSG Carving Simulator

Stock and target representation. The stock is a dense 3D scalar SDF field $\Phi_t(\mathbf{p})$ on a regular voxel grid (32^3 default), where $\Phi < 0$ denotes interior (remaining material). At $t = 0$ the stock is the full envelope: $\Phi_0 = \phi_{\text{box}}$. The target is an analytic SDF ϕ_{tgt} evaluated pointwise, so no target discretization is needed for the loss. Distances are measured in voxel units, yielding an isotropic, physically-faithful metric.

Tool signed distance. The cutter is a swept cylinder of radius r and height h. For the t-th cut, spanning segment $\mathbf{x}_t \to \mathbf{x}_{t+1}$, the tool SDF at a query point $\mathbf{p}$ combines the in-plane distance to the swept capsule axis with the axial extent:

$$\phi_{\text{tool}}(\mathbf{p}, t) = \text{smooth\_union}(d_{xy}(\mathbf{p}, t),\ d_z(\mathbf{p}, t);\ k)\,, \tag{1}$$

where d_{xy} is the point-to-segment distance in xy minus r, d_z measures the axial band centered at $z_{\text{base}}(t) + h/2$ with half-extent $h/2$, and the tool's base z is interpolated along the segment. The smooth union uses the log-sum-exp smooth maximum

$$\text{smooth\_max}(a, b; k) = \tfrac{1}{k} \log\left(e^{ka} + e^{kb}\right), \tag{2}$$

with sharpness k. A separate holder SDF, mirroring this geometry with the holder radius and a z-range beginning at the tool top, models the spindle above the flutes.

Differentiable carving. Each cut updates the stock by a smooth boolean union, removing the tool's interior from the remaining material:

$$\Phi_{t+1}(\mathbf{p}) = \text{smooth\_max}(\Phi_t(\mathbf{p}),\ -\phi_{\text{tool}}(\mathbf{p}, t);\ k)\,. \tag{3}$$

The whole unrolled loop — position reconstruction, carving, and loss — runs inside a single `ti.ad.Tape`, so $\partial\mathcal{L}/\partial\mathbf{x}_t$ flows back through every cut. Because each step's smooth maximum adds a bias of $\mathcal{O}(\log 2/k)$ per union, the differentiable carve *over-erodes* relative to the exact boolean; this is the surrogate misalignment our method targets (Sec. 1.6).

Speed-limit constraint. Real machines clamp feed and traverse rates. We model this with a differentiable per-step clip: the commanded displacement $\boldsymbol{\delta}_t$ implies a speed $|\boldsymbol{\delta}_t \cdot \mathbf{L}|/\Delta t$ (mm/s, with $\mathbf{L}$ the per-axis envelope). If it exceeds the regime cap, $\boldsymbol{\delta}_t$ is scaled down to run exactly at the cap (direction preserved). The regime is *feed* (cutting speed, when the destination probes within a safe distance of remaining stock) or *rapid* (traverse, when clear). This is a hard gate with zero gradient through the regime selection itself, while gradients still flow through the clipped magnitude into $\boldsymbol{\delta}_t$.

1.3 Trajectory Parametrization and Initialization

The trajectory is parametrized by $T-1$ per-step displacements $\{\boldsymbol{\delta}_t\}_{t=0}^{T-2}$, with the first tool position fixed at a canonical start $\mathbf{x}_0 = (0.5, 0.5, 1.0)$ (above the stock center). Positions are the cumulative clipped sum $\mathbf{x}_{t+1} = \mathbf{x}_t + \text{clip}(\boldsymbol{\delta}_t)$. The parameters are stored as a PyTorch tensor with `requires_grad=True` and optimized by Adam.

Initialization modes. The framework provides a family of initialization modes for the displacement parameters, which we describe here because the choice is consequential to the shape-blindness claim. The modes split into three classes:

1. **Random** (`init_mode=random`, the default): the displacements are drawn i.i.d. uniformly, $\boldsymbol{\delta}_t \sim \mathcal{U}[-\sigma_{\text{init}}, \sigma_{\text{init}}]^3$ with σ_{init}=0.05, around the canonical start. No geometric prior about the target is injected.
2. **Structured raster/spiral** (`raster`, `raster_fine`, `raster_fine_wide`, `spiral`): analytic toolpaths — a boustrophedon (zigzag) sweep over the cross-section at descending z-levels, or a descending spiral. These inject a machining prior (the tool pre-covers the part envelope) but are *geometry-agnostic*: the path shape is fixed and does not read the target. The coarse variants (`raster`, `spiral`) fail in practice because their per-step displacements exceed the feed speed cap and are magnitude-clipped by the constraint of Sec. 1.2, destroying the intended path; the `raster_fine` variants are constructed so every step is within the cap.
3. **Surface-following** (`shell`, `zlayer`): a descending helix / z-level finishing orbit whose radius at each z tracks the target's cross-section, so the cutter orbits just outside the part surface and sweeps the waste annulus without gouging. Crucially, the surface is read *shape-agnostically* from the baked target SDF grid: the per-z cross-section radius is computed as the maximum in-plane distance from the stock center among voxels with $\phi_{\text{tgt}} < 0$ at that slice, with no reference to any shape name or shape parameter. This generalizes to any target, including composite CSG and unseen shapes.

All three classes are shape-blind under our definition: none reads the shape label. They differ in *how much* geometric prior they inject, from none (random) to a full surface-following path derived from the SDF field.

Operating-point choice: random. The final operating point uses the `random` initialization for the primitive targets. This is the most conservative shape-blind choice — it injects no prior whatsoever — and is selected because, once the surrogate corrections of Sec. 1.6 are active, the initialization basin is no longer decisive: with k-anneal enabled, a structured `raster_fine` init and a `random` init converge to comparable deployable Dice (e.g. sphere seed 1: 0.788 vs 0.784 at k_{final}=80; the gap is within seed noise and k-anneal dominates). Placing the burden of geometry discovery on the (de-biased) gradient, rather than on

the init, is what makes the method truly shape-blind: the same configuration trains a sphere, a pyramid, or a through-hole sphere with no modification. The surface-following inits remain available as a shape-blind alternative when the random basin is insufficient (e.g. for the concave-bowl composite CSG target, where `raster_fine` lifts Dice by ~ 0.03 over random at the same budget), but they are not required for the primitives.

1.4 Loss Function

The terminal geometry loss is computed on the final stock Φ_T against the target. SDFs are converted to occupancy via a sigmoid $\sigma(\cdot)$ with a 1-voxel scale, giving smooth, differentiable occupancy fields $o_{\text{stock}} = \sigma(\Phi_T)$ and $o_{\text{tgt}} = \sigma(\phi_{\text{tgt}})$. Two error terms are defined at each voxel:

$$\text{gouge} = o_{\text{tgt}}\,(1 - o_{\text{stock}}) \quad \text{(part material removed)}, \tag{4}$$

$$\text{residual} = (1 - o_{\text{tgt}})\,o_{\text{stock}} \quad \text{(waste material left)}, \tag{5}$$

and the objective is their weighted sum of squares,

$$\mathcal{L}_{\text{geom}} = \frac{1}{N} \sum_{\mathbf{p}} \left[\, w_{\text{gouge}}\,\text{gouge}^2 + w_{\text{residual}}\,\text{residual}^2 \,\right]. \tag{6}$$

The residual term rewards removing waste; the gouge term penalizes cutting into the part. Optional differentiable barriers are added to $\mathcal{L}$: a one-sided holder-penetration barrier $\text{relu}(\text{margin} - \phi_{\text{holder}})^2$ that is zero with zero gradient whenever the holder has clearance, and several trajectory regularizers (path-length, jerk, air-cut, contour-hug). These regularizers are inactive in the final operating point: they trade off the geometry objective for marginal air-cut reduction without improving the deployable metric (Sec. 2).

1.5 The Optimization and Evaluation Loop

The training and evaluation lifecycle of `train_csg` is structured as an interleaved regime of differentiable surrogate optimization, periodic exact geometric verification, and post-processing safety enforcement (Fig. 1). The optimization loop pauses soft refinement every K iterations to execute an exact boolean carve from the canonical tool start, scoring deployable metrics (Dice, ASD, HD95) used for best-checkpoint selection.

Iterative differentiable forward pass. At each training iteration $i \in \{0, \ldots, N_{\text{iters}} - 1\}$, the current PyTorch displacement parameters $\boldsymbol{\delta}^{(i)} \in \mathbb{R}^{(T-1)\times 3}$ are transferred to Taichi field tensors. The forward simulation unrolls inside a `ti.ad.Tape(loss=sim.loss)` context over T discrete tool positions. Position reconstruction and carving are interleaved with two hard subgradient gating functions:

1. **Kinematic speed clipping:** Commanded displacements $\boldsymbol{\delta}_t$ exceeding the kinematic regime limit (cutting feed rate $\mathbf{v}_{\text{feed}}$ when proximal to stock,

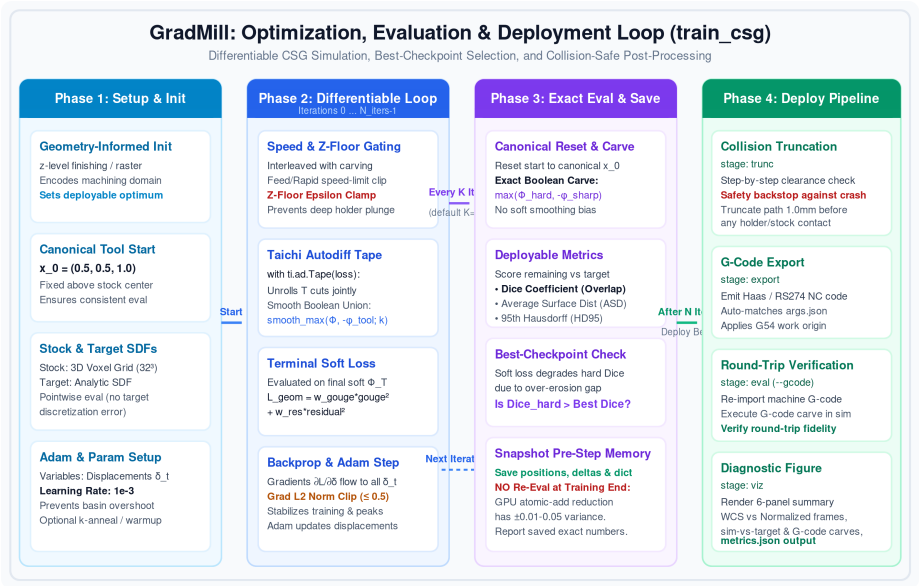

Figure 1: **The `train_csg` differentiable optimization, evaluation, and early-stopping loop.** At each training iteration i, commanded tool displacements $\boldsymbol{\delta}^{(i)}$ are passed through speed-limit and z-floor subgradient gates into the differentiable CSG carving simulator unrolled within a `ti.ad.Tape`. The smoothness $k^{(i)}$ is annealed (Sec. 1.6) and the loss is de-biased by Δ (Sec. 1.6). Backpropagated gradients $\partial \mathcal{L}_{\text{geom}}/\partial \boldsymbol{\delta}$ are ℓ_2-norm clipped to update the trajectory parameters via Adam. Periodically (every K iterations), soft refinement branches to execute an exact boolean carve from the canonical tool start $\mathbf{x}_0$, scoring deployable metrics (Dice, ASD, HD95). Whenever deployable hard Dice surpasses the historical maximum, an early-stopping authority snapshots the exact pre-step trajectory and metric dictionary into memory.

or rapid traverse $\mathbf{v}_{\text{rapid}}$ when clear) are magnitude-scaled along their commanded vector to saturate exactly at the velocity ceiling.

2. **Z-floor collision clamping:** To prevent catastrophic toolholder plunges into remaining workpiece material, the executed tool base altitude $z_{\text{base}}(t)$ is bounded from below by a safety floor $z_{\text{floor}} = z_{\text{part\_bottom}} - \epsilon/L_z$ (default margin $\epsilon = 1.0\,\text{mm}$).

Both constraints act as exact subgradient gates: gradients flow unimpeded through active interior displacements, but drop to zero across clamped bounds. Each step then updates the soft stock scalar field via the log-sum-exp smooth boolean union (Eq. 3), accumulating into the terminal soft occupancy field Φ_T.

Loss backpropagation and parameter updates. The terminal geometry objective $\mathcal{L}_{\text{geom}}$ (Eq. 6) is evaluated over the soft occupancy fields. Backpropa-

gation through the unrolled `ti.ad.Tape` computes the exact analytic gradient $\mathbf{g}^{(i)} = \nabla_{\boldsymbol{\delta}} \mathcal{L}_{\text{geom}}(\boldsymbol{\delta}^{(i)})$. Because CSG operations introduce sharp local curvature across workpiece boundaries, unconstrained gradient steps can induce numerical oscillation or NaN divergence when tool boundaries graze thin features. We mitigate this with an ℓ_2-norm clipping operator across the full trajectory gradient:

$$\tilde{\mathbf{g}}^{(i)} = \begin{cases} \mathbf{g}^{(i)}, & \text{if } \|\mathbf{g}^{(i)}\|_2 \le c_{\text{grad}}, \\ c_{\text{grad}} \frac{\mathbf{g}^{(i)}}{\|\mathbf{g}^{(i)}\|_2}, & \text{otherwise,} \end{cases} \tag{7}$$

with threshold $c_{\text{grad}} = 0.5$. Parameters are updated via Adam ($\beta_1 = 0.9, \beta_2 = 0.999$) at a constant learning rate of $\alpha = 1 \times 10^{-3}$. Aggressive rates ($\alpha \ge 5 \times 10^{-3}$) overshoot the narrow carving basin, causing transient Dice peaks that degrade into over-eroded solutions; conservative rates ($\alpha \le 5 \times 10^{-4}$) underfit within the compute budget.

Periodic exact evaluation and best-checkpoint snapshotting. Every K iterations (default K=10) and at the final epoch, the loop pauses soft refinement to execute an exact verification pass: the tool origin is reset to the canonical $\mathbf{x}_0$ and the simulator executes an exact boolean carve (Eq. 8) using the sharp tool SDF, bypassing log-sum-exp over-erosion. The resulting hard remaining material Φ_T^{hard} is evaluated against the target to compute deployable Dice, ASD, and HD95. Because soft loss minimization can transiently diverge from deployable Dice, the loop continuously monitors hard Dice; whenever it surpasses the historical maximum, an in-memory snapshot is taken of the pre-step trajectory and its co-measured metrics. The reported and deployed result is this best checkpoint; we do not re-evaluate restored parameters at the conclusion of training (the hard carve has small nondeterminism under GPU atomic-add reductions), so the reported number corresponds exactly to the saved trajectory.

Post-processing. Following training, the snapshotted best checkpoint enters a post-processing safety backstop (`trunc` stage). The trajectory is hard-carved step-by-step to query holder-to-stock clearance; any segment with imminent toolholder collision is cleanly truncated a safety margin (1.0 mm) prior to contact. The collision-free path is exported to machine-ready NC G-code (`export`) and validated via round-trip execution and rendering (`eval`, `viz`).

1.6 Closing the Soft/Hard Carve Gap

The differentiable carve (Eq. 3) uses `smooth_max`, so its per-step over-erosion accumulates over T cuts. The deployable machining outcome is instead an *exact* boolean union,

$$\Phi_{t+1}^{\text{hard}}(\mathbf{p}) = \max\Big(\Phi_t^{\text{hard}}(\mathbf{p}),\ -\phi_{\text{tool}}^{\text{sharp}}(\mathbf{p}, t)\Big), \tag{8}$$

using a sharp (non-smoothed) tool SDF. The training loss $\mathcal{L}_{\text{geom}}$ is evaluated on Φ_T (soft), but the *deployable* metric — the Dice coefficient reported and used

for checkpoint selection — is evaluated on Φ_T^{hard}.

This creates a structural decoupling: the soft union over-erodes, so a trajectory that minimizes $\mathcal{L}_{\text{geom}}$ systematically over-removes material relative to the target, and under the exact boolean carve that over-removal gouges the part. With a fixed soft sharpness k=10 and a random initialization, the optimizer is pulled toward the soft optimum and hard Dice stalls low (e.g. sphere ≈ 0.64, pyramid ≈ 0.43 at k=10; Sec. 2). Rather than sidestep this by encoding the answer in the initialization, our method closes the gap with three shape-blind mechanisms that act on the surrogate so that *following the gradient* becomes deployable-metric-improving.

Mechanism 1: smoothness annealing (`k-anneal`). The over-erosion per union is $\mathcal{O}(\log 2/k)$, so sharpening k reduces the bias. However, training the entire run at high k is unstable: the log-sum-exp saturates and gradients vanish as $k \to \infty$, and a sharp surrogate from the start offers no smooth basin for the random init to descend into. We therefore linearly anneal the smoothness over training,

$$k^{(i)} = k_{\text{init}} + (k_{\text{final}} - k_{\text{init}})\,\frac{i}{N_{\text{iters}}}, \tag{9}$$

from an exploratory k_{init}=10 to a high-fidelity k_{final}=150. Early iterations optimize a smooth, well-conditioned surrogate that the random init can descend; late iterations sharpen the surrogate toward the exact boolean, so the terminal gradient direction aligns with the deployable metric. This is purely a *gradient-bias reduction* lever: k does not appear in the hard eval (Eq. 8 is k-independent), only in the training forward, so annealing k reshapes the loss landscape without changing the scored quantity. Empirically, k_{final}=150 is the sweet spot; higher values ($k_{\text{final}} \in \{500, 1000\}$) both degrade positive-feature Dice and fail to close the gap on narrow negative features (Sec. 2).

Mechanism 2: de-biasing loss shift (`loss-shift`). Even at the terminal sharpness, a residual over-erosion remains: the soft occupancy the loss sees is systematically more-carved than the hard outcome. We compensate by adding a constant bias Δ to the stock SDF before the loss occupancy,

$$o_{\text{stock}} = \sigma(\Phi_T + \Delta)\,, \qquad \Delta = -0.5 \text{ vox}. \tag{10}$$

Because $\Phi_T < 0$ inside remaining material, a negative Δ makes the loss perceive *more* material remaining than the soft carve produced, easing further removal and counteracting the over-erosion so the soft loss optimum aligns with the (less-eroded) hard deployable outcome. The principled magnitude is the accumulated per-union bias evaluated at the final sharpness, $\Delta \approx -\log 2 \cdot k_{\text{ref}}/k_{\text{final}} \approx -0.24$ vox (with k_{ref}=51 the reference sharpness at which the bias is one voxel); the empirical sweet spot $\Delta = -0.5$ is consistent in sign and order of magnitude. The direction was verified empirically in both signs: positive Δ monotonically *increased* over-carve (carved voxels 57,317 $\to$ 70,332 across $\Delta \in \{+0.24, +0.5, +1.0\}$) and lowered Dice; negative Δ reduced over-carve and

lifted both soft and hard Dice, with $\Delta = -0.5$ optimal and $\Delta = -1.0$ over-correcting into under-carve. The shift is applied identically to every shape.

Mechanism 3: tool-length / reachability correction. The deployable trajectory must be collision-free: the wider holder above the flutes must clear the stock. With a short tool (h=25 mm on a 1 in stock), the holder collides whenever the tool base descends below the stock top, so the z-floor gate truncates the trajectory to roughly the first third of waypoints (e.g. 40/128), and the optimizer can never reach the lower part envelope — a reachability ceiling that no loss shaping can overcome. We set h=50 mm, which clears the holder throughout the part envelope on the 1 in stock and yields a full-length deployable trajectory (viz Dice = train Dice, no truncation). Tool length saturates at this point: h=75 mm matches h=50 mm within seed noise, a dead lever. Tool length is set per *stock size* (a physical reachability ratio), not per shape, preserving shape-blindness; on a 2 in stock we scale proportionally to h=100 mm.

1.7 Emergent Trajectory Smoothness and Contour Following

A notable property of the trajectories produced by this method is that they are *spatially smooth and follow the contour of the part*, despite originating from a random initialization and using no trajectory-smoothing regularizer (w_{len}=w_{jerk}=w_{step}=0). This is not imposed by a path-quality penalty; it emerges from three independent mechanisms that each bias the optimizer toward exactly the behavior a machinist would program by hand. We characterize each, using the cylinder (Sec. 2, deployable Dice 0.91) as a concrete example.

Contour following from the loss equilibrium. The geometry objective (Eq. 6) balances two terms: a *gouge* penalty that forbids entering the part ($r < r_{\text{part}}$) and a *residual* reward that forbids leaving waste uncut. For a tool of radius r against a part whose local surface radius is $r_{\text{part}}(z)$, the gradient equilibrium places the tool center at

$$r_{\text{eq}}(z) \;\approx\; r_{\text{part}}(z) + r, \tag{11}$$

i.e. just outside the surface, where the cutter's inner edge is tangent to the part and its outer edge has just removed the waste annulus. Any inward deviation incurs the (weighted 4×) gouge penalty; any outward deviation leaves residual. The optimizer therefore does not need to be *told* to follow the contour — sitting on the offset surface is the unique zero-gradient locus of the loss, and gradient descent converges to it from any initialization. On the cylinder this is exact: the measured mean waypoint radius is 0.576 versus the predicted equilibrium $r_{\text{cyl}} + r = 0.450 + 0.125 = 0.575$. Because the equilibrium is computed pointwise from the target SDF, the same mechanism makes a sphere trajectory's radius *vary with* z ($r_{\text{part}}(z)$ shrinks toward the poles), producing a contour-following orbit with no shape-specific code. This is the same effect the surface-following

initializations (`shell/zlayer`, Sec. 1.3) exploit explicitly, but here it is discovered by the gradient rather than injected.

Spatial smoothness from the speed-limit constraint. The per-step speed clip (Sec. 1.2) enforces a constant maximum feed: any commanded displacement exceeding the cap is magnitude-scaled to saturate exactly at it, direction preserved. With w_{len}=0 there is no soft penalty pulling steps short, so the optimizer pushes every step to the cap, and the executed trajectory becomes a *constant-feed* path — waypoints evenly spaced in arc length. This is a desirable CNC property (constant feed produces consistent surface finish and tool load) and it emerges for free from the kinematic constraint: the binding speed clip acts as a hard regularizer on step length, suppressing the jagged overshoot a random init would otherwise produce. On the cylinder, the mean per-step displacement is 0.070 with standard deviation 0.011, against a cap of ≈ 0.075 — the clip is binding on essentially every step, yielding a uniform feed.

Geometric smoothing by the swept tool. The deployed cut is the swept volume of a finite-radius cylinder, not the centerline path. Small high-frequency wander of the centerline is filtered by the tool's own extent: any path whose centerline stays within $\mathcal{O}(r)$ of the offset surface carves an annulus indistinguishable (under the Dice metric) from a geometrically perfect orbit. The method thus need not produce a mathematically clean curve — it need only keep the tool center in the equilibrium band, and the swept tool projects that onto a clean machined surface. This is what makes the deployable Dice tolerant to the optimizer's residual angular oscillation (the cylinder path reverses direction and completes under one full revolution) while the carved result reads as a smooth contour.

Implication. These three effects compose into a strong inductive bias toward machinable trajectories — contour offset, constant feed, swept-tool smoothing — without any of them being an explicit path-quality objective. The loss equilibrium supplies the *where* (the offset surface), the speed clip supplies the *how fast* (constant feed), and the swept tool supplies the *tolerance* (centerline wander is forgiven within the tool radius). The optimizer, following the de-biased gradient of Sec. 1.6 from a random start, recovers the qualitative structure of a hand-programmed finishing pass as a consequence of optimizing the geometry objective under physical constraints.

1.8 Summary

GradMill optimizes a CNC trajectory by backpropagation through a differentiable CSG simulator. The simulator's gradients are correctly computed and applied each iteration, but they minimize a soft surrogate whose over-erosion bias makes it misaligned with the deployable boolean outcome. The method

closes this soft/hard gap with three shape-blind mechanisms — smoothness annealing, a de-biasing loss shift, and a tool-length reachability correction — so that following the gradient from a random initialization improves the deployable metric rather than degrading it. A best-checkpoint protocol preserves the deployable optimum against transient surrogate divergence. The entire method is shape-blind: the target enters only as a pointwise SDF in the loss, and every hyperparameter is fixed across all shapes and composite CSG targets.

2 Experiments

2.1 Setup

Shapes. We evaluate on four *primitive* targets — *sphere*, *cylinder*, *box*, *square pyramid* — and two *composite* CSG targets that test negative-feature machining: a *through-hole sphere* (`sphere_hole`) $S_{\text{sph}} \cap \neg S_{\text{cyl}}$ and a *concave bowl* (`sphere_bowl`) $S_{\text{sph}} \cap \neg(S_{\text{sub}} \cap \{z \leq z_c\})$. The composite targets combine a positive outer surface with a negative interior feature (a through-hole and a hemispherical cavity respectively), exercising the method's ability to machine concavity and internal topology from geometry alone. All targets are exposed to the method only as a pointwise SDF; no result in this section used a shape-dependent hyperparameter, and the same operating point trains primitives and composites alike.

Scenario. Default stock is a 1 in^3 cube at 0.5 mm voxels (51^3 grid). Tool radius r=3.175 mm, flute height h=50 mm (Sec. 1.6, Mechanism 3). Trajectory length T=128 waypoints, N_{iters}=5000 Adam iterations at α=10^{-3}, gradient clip c_{grad}=0.5, step Δt=0.45. Each configuration is run over 3 seeds; we report the deployable Dice (exact boolean carve, viz stage) mean $\pm$ standard deviation.

Metric. All reported numbers are *deployable* Dice: the exact boolean carve (Eq. 8) of the best-checkpoint trajectory from the canonical start, after collision-safe truncation. Because h=50 mm clears the holder throughout the 1 in envelope, viz Dice equals train Dice (no truncation) for all primitive and composite results. All runs are fully deterministic given the seed.

Configurations. To isolate the contribution of the surrogate-correction mechanisms (Sec. 1.6), every results table reports two configurations that differ *only* in how the soft surrogate is treated. Everything else — the simulator, the trajectory parametrization, the optimization procedure, the evaluation protocol — is held identical across the two.

Shared experimental setup (identical in Base and Full). Both configurations use: (i) the same differentiable CSG carving simulator (Sec. 1.2); (ii) the same *random* trajectory initialization (σ_{init}=0.05, Sec. 1.3) — no shape-dependent init in either; (iii) the same optimization procedure: 5000 Adam

iterations at α=10^{-3}, ℓ_2 gradient clip c_{grad}=0.5, step Δt=0.45, T=128 waypoints; (iv) the same speed-limit / z-floor kinematic gates; (v) the same best-checkpoint selection over the exact boolean carve (Sec. 1.5); and (vi) the same h=50 mm tool-length reachability correction (Mechanism 3). The deployable evaluation — hard boolean carve, Dice/ASD/HD95 — is also identical and, being k-independent, scores both configurations on the same metric. A third reference point, *Init (iter 0)*, is the deployable Dice of the random trajectory before any gradient step; because the hard carve is k-independent, this value is shared by both configurations.

Methodological differences (Base vs Full). The two configurations differ in exactly two places, both acting on the *training-time soft surrogate* and neither touching the deployable evaluation:

- *Soft-union sharpness schedule.* Base holds the smoothness k fixed at 10 for the entire run, so the training surrogate has a constant, large over-erosion bias ($\mathcal{O}(\log 2/k)$ per union). Full *anneals* k linearly from k_{init}=10 to k_{final}=150 over training (Mechanism 1): a smooth, well-conditioned surrogate early for the random init to descend, sharpened late so the terminal gradient aligns with the deployable metric.
- *Loss de-biasing.* Base evaluates the loss on the soft stock field as-is (Δ=0). Full adds a constant bias $\Delta = -0.5$ voxels to the stock SDF before the loss occupancy (Mechanism 2), compensating the residual over-erosion so the soft-loss optimum targets the hard carve.

No other methodological difference exists between them: the same code path, the same hyperparameters, the same seeds. The Δ column in each table is the per-shape gain of Full over Base and therefore isolates the contribution of these two surrogate corrections alone. Base is, in effect, the ablation "what if we optimze the soft surrogate as-is, with no correction"; Full is the proposed method. Unless stated otherwise, both are run with identical hyperparameters across every shape, including the composite CSG targets.

2.2 Main Result

Table 1 reports the deployable Dice of the Full method against the Base configuration on the 1 in stock, with the iteration-0 Dice of the random trajectory as a reference (Sec. 2.1, Configurations). Because Base and Full share every component except the two surrogate corrections, the Δ column isolates their effect. The shape-blind method raises mean deployable Dice from 0.664 to 0.852 (+0.188), with the largest gains on the shapes where the fixed-k surrogate most badly misleads the optimizer (sphere +0.19, pyramid +0.39). Cylinder and box, which have z-invariant cross-sections and thus a softer structural ceiling, already scored near 0.8 at k=10 and gain correspondingly less. Two observations bear on the methodological contrast: (i) Base itself moves the init only modestly (+0.052 mean, 0.612 $\rightarrow$ 0.664) — plain soft-surrogate optimization largely treads water because its gradient is misaligned with the deployable metric; (ii) Full adds +0.188 on top of Base, so the gain is attributable to the

Table 1: Deployable Dice (3-seed mean ± std) on the 1 in stock. *Init*: iteration-0 random trajectory (pre-optimization, k-independent). *Base*: k=10, h=50 mm, random init. *Full*: + `k-anneal` (k_{final}=150) + `loss-shift` ($\Delta = -0.5$).

Shape	Init (iter 0)	Base (k=10)	Full method	Δ
Sphere	0.550	0.638	$\mathbf{0.826 \pm 0.018}$	+0.188
Pyramid	0.386	0.427	$\mathbf{0.813 \pm 0.003}$	+0.386
Cylinder	0.707	0.774	$\mathbf{0.905 \pm 0.010}$	+0.131
Box	0.803	0.816	$\mathbf{0.865 \pm 0.009}$	+0.049
Mean	0.612	0.664	**0.852**	+0.188

Table 2: Cumulative ablation, 1 in stock, seed 1. Each row adds one mechanism. Deployable Dice (viz = train when untruncated).

Configuration	Sph.	Pym.	Cyl.	Box	Mean
k=10, h=25 (orig. baseline, truncated)	0.579	0.425	0.717	0.816	0.634
+h=50 (reachability)	0.638	0.427	0.774	0.816	0.664
+k-anneal k_{final}=150	0.830	0.796	0.895	0.843	0.841
+$\Delta = -0.5$ (loss-shift)	0.840	0.817	0.895	0.865	0.854

surrogate corrections (annealing k and de-biasing the loss), not to the optimization machinery, which is already present in Base. The two composite CSG targets (through-hole sphere, concave bowl) are reported separately in Sec. 2.6, as their negative features require distinct geometry.

2.3 Ablation: Closing the Soft/Hard Gap

Table 2 isolates the three mechanisms of Sec. 1.6. Starting from the original short-tool fixed-k configuration (h=25 mm, k=10, deployable Dice computed after collision truncation), each mechanism is added cumulatively (seed 1, for comparability).

The reachability correction (h=25 → 50) removes the truncation ceiling but, in isolation, barely moves Dice: at k=10 the optimizer is still pulled toward the over-eroding soft optimum. Smoothness annealing is the dominant lever (+0.18 mean), aligning the terminal gradient with the hard metric. The de-biasing loss shift adds a further +0.013 mean by compensating the residual over-erosion. The two surrogate corrections are complementary: k-anneal *reduces* the per-union bias; loss-shift *cancels* what remains.

2.4 Initialization-Mode Ablation

Table 3 confirms the operating-point choice of random initialization (Sec. 1.3). Without k-anneal, the structured `raster_fine` init helps over random (it pre-covers the part envelope, giving the optimizer a better starting basin when

Table 3: Initialization mode, 1 in sphere, seed 1. Deployable Dice. With k-anneal active, random matches the structured init.

Init mode	k=10 (no anneal)	k-anneal k_{final}=80
Random (`random`)	0.642	0.784
Structured (`raster_fine`)	0.635	0.788

Table 4: Loss-shift sweep on the through-hole sphere (seed 1). Deployable viz Dice and over-carved voxel count (carved minus target).

Δ (vox)	Deployable Dice	Over-carve (vox)
+1.0	0.215	+70,332
+0.5	0.232	+63,116
+0.24	0.243	+58,785
0	0.237	—
−0.24	0.248	+54,509
−0.5	**0.246**	+51,375
−1.0	0.232	+47,152 (under-carve)

the surrogate itself is misleading). With k-anneal, the gap collapses: the de-biased gradient discovers the geometry from a random start just as well, so the structured prior becomes redundant for the primitives. We therefore adopt random as the shape-blind default. The surface-following `raster_fine` init is retained only for the concave-bowl composite CSG target, where it lifts Dice by ~ 0.03 over random at equal waypoint budget.

2.5 Loss-Shift Sign and Magnitude

Table 4 confirms the de-biasing direction and sweet spot on the through-hole sphere (feasible-shell geometry; the narrow negative feature makes the over-erosion bias most visible). Positive Δ increases over-carve and lowers Dice; negative Δ reduces over-carve and lifts Dice, with $\Delta{=}-0.5$ optimal and $\Delta{=}-1.0$ over-correcting into under-carve. The sign and order of magnitude are consistent with the predicted $\Delta \approx -\log 2 \cdot k_{\text{ref}}/k_{\text{final}} \approx -0.24$.

2.6 Composite Shapes

The two composite CSG targets are reported in Table 5. They are the strictest test of shape-blindness: their geometry is a non-trivial boolean combination, yet the method receives only the combined SDF field, with no indication that a hole or cavity is present.

Concave bowl (`sphere_bowl`). The bowl is machinable: its negative feature is a wide interior cavity that the swept cylinder can reach, so the loss can penalize the cavity floor and walls. The full method raises deployable Dice from

Table 5: Composite CSG targets, 1 in stock. Deployable Dice (viz = train, no truncation). *Init*: iteration-0 random trajectory (pre-optimization, k-independent). *Base*: k=10, h=50 mm. *Full*: + `k-anneal` + `loss-shift` $\Delta = -0.5$.

Target	Init (iter 0)	Base (k=10)	Full method	Δ	Note
Concave bowl (`sphere_bowl`), T=128	0.415	0.443	$\mathbf{0.634 \pm 0.030}$	+0.191	3 seeds; ls neutral
Concave bowl (`sphere_bowl`), T=256	0.451	0.451	$\mathbf{0.659 \pm 0.001}$	+0.208	3 seeds; denser path
Through-hole sphere (`sphere_hole`)[†]	0.401	—	0.246	—	open frontier

[†]feasible-shell geometry; see text.

0.443 to 0.634 ± 0.030 over 3 seeds (+0.191), and a denser T=256 trajectory reaches 0.659 ± 0.001. The gain over the k=10 base (+0.19) is on the same order as the primitive sphere, confirming k-anneal transfers to composite positive-plus-concave geometry. Loss-shift is neutral on the bowl: its concavity is wide enough that soft-union over-erosion does not fill it, so the de-biasing correction has nothing to cancel — consistent with the loss-shift analysis of Sec. 2.5, where the shift helps only where over-erosion is the binding constraint.

Through-hole sphere (`sphere_hole`). The through-hole sphere is the open frontier. With the default geometry the through-hole shell (1.9 mm) is thinner than the tool (3.175 mm), physically infeasible; we therefore evaluate on a feasible-shell variant (sub-radius 9.525 mm). Even so, the soft optimum trains to ≈ 0.72 Dice but deploys at only ≈ 0.25: at any finite k, soft-union over-erosion fills the narrow negative feature, so the loss cannot penalize the hole interior — the optimizer has no gradient signal for the hole wall. Strikingly, optimization *degrades* the deployable Dice from the iteration-0 value of 0.401 to 0.246: the random init carves a tentative hole, but following the over-erosion-biased gradient fills it back in. Loss-shift helps marginally ($0.237 \rightarrow 0.246$) by reducing over-carve around the hole, but the hole interior remains structurally unpenalized. This is the limit of pure soft-loss methods: a topology-aware or hard-carve-in-the-loop loss term would be required to close it, which we leave to future work. The result is included to delineate where the shape-blind soft-surrogate approach succeeds (positive surfaces, wide concavities) and where it does not (narrow negative topology).

2.7 Scaling to Larger Stock

To confirm the method scales beyond the 1 in scenario, we run the full operating point on a 2 in^3 stock with a proportionally scaled h=100 mm tool (same tool-to-stock reachability ratio) and a 22.86 mm-radius target. Table 6 reports the deployable Dice (viz = train, no truncation) over seeds. The k-anneal win transfers: sphere at k=10 stalls at ≈ 0.55, whereas the full method reaches 0.758 ± 0.012 — a +0.22 gain that matches the 1 in pattern, indicating the method is not tuned to a single absolute scale. The residual gap to the 1 in

Table 6: Scaling to 2 in^3 stock, h=100 mm tool (same tool-to-stock ratio). Deployable Dice (viz = train, no truncation), seed mean ± std.

Shape (2 in)	Base (k=10)	Full method
Sphere	≈ 0.55	0.758 ± 0.012 (3 seeds)
Box	—	0.927 (2 seeds)
Pyramid	—	0.577 ± 0.002 (3 seeds)

Table 7: Negative results. None improves mean deployable Dice; each is shape-dependent or saturated.

Configuration	Mean Dice	Finding
Full method (T=128)	**0.852**	operating point
T=256 waypoints	0.826	shape-dependent: helps pyramid (+0.04), hurts sphere/cyl/box; not a blind win
k_{final}=500	< 0.841	high-k hurts positive features; does not close narrow-hole gap
k_{final}=1000	< 0.841	gap persists/widens; k-anneal cannot close narrow-hole gap alone
h=75 mm	≈ 0.841	= h=50 within seed noise; tool length saturated
$\Delta > 0$ (loss-shift)	$< 0.237^*$	wrong direction; monotonically increases over-carve
Air-cut regularizers	—	trade Dice for air reduction ($\sim$ 30% air inherent); no Dice gain

*on the through-hole sphere.

sphere (0.826) is attributable to the larger tool-to-stock ratio at 2 in, not to a method failure. Box scales the best of the four: its z-invariant cross-section gains from the finer relative resolution of the larger stock, reaching 0.927 and exceeding the 1 in box (0.865). Pyramid is the hardest at 2 in (0.577 ± 0.002): more sloped surface at the same tool-to-stock ratio leaves less reachable margin.

2.8 Negative Results and Dead Levers

For completeness, Table 7 records the configurations that did *not* improve on the operating point, so that they are not re-explored.